Lesson 1.01 **Duration: 40 minutes**

Functions Introduction

Learning Objectives

- Define what a function is and explain its purpose in programming
- Identify the components of a function: name, parameters, and return value
- Write simple functions that perform specific tasks

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Understanding functions as mathematical mappings
Science: Functions organize code like scientific experiments
Language: Writing function documentation builds technical writing

Digital Citizen Reminder: Apply Functions Introduction skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Functions Introduction support
On Level: Demonstrate Functions Introduction through practical exercises
Advanced: Apply Functions Introduction to solve complex problems independently
SEND: Practice Functions Introduction with step-by-step teacher guidance

Formative Assessment

Method: Practical Functions Introduction activity
CT Skill Assessed: Decomposition - breaking down Functions Introduction into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Functions Introduction. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.02 **Duration: 40 minutes**

Function Calls

Learning Objectives

- Distinguish between function definition and function invocation
- Pass arguments to functions and capture return values
- Trace the execution flow of a program with multiple function calls

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Calling functions uses parameters - variable substitution
Science: Function return values connect to mathematical outputs
Language: Writing clear function calls builds precise coding skills

Digital Citizen Reminder: Apply Function Calls skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Function Calls support
On Level: Demonstrate Function Calls through practical exercises
Advanced: Apply Function Calls to solve complex problems independently
SEND: Practice Function Calls with step-by-step teacher guidance

Formative Assessment

Method: Practical Function Calls activity
CT Skill Assessed: Decomposition - breaking down Function Calls into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Function Calls. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.03 **Duration: 40 minutes**

Arrays Concept

Learning Objectives

- Explain what an array is and why it is useful for storing multiple values
- Access and modify elements in an array using index positions
- Describe zero-based indexing and how it applies to arrays

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Arrays store sequential data - connecting to number sequences
Science: Array indexing uses zero-based counting
Language: Writing array operations builds data structure skills

Digital Citizen Reminder: Apply Arrays Concept skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Arrays Concept support
On Level: Demonstrate Arrays Concept through practical exercises
Advanced: Apply Arrays Concept to solve complex problems independently
SEND: Practice Arrays Concept with step-by-step teacher guidance

Formative Assessment

Method: Practical Arrays Concept activity
CT Skill Assessed: Decomposition - breaking down Arrays Concept into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Arrays Concept. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.04 **Duration: 40 minutes**

Array Operations

Learning Objectives

- Perform common array operations: insert, delete, search, and sort
- Analyze the efficiency of different array operations
- Implement algorithms that manipulate arrays to solve problems

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Array methods perform operations on collections - set theory
Science: Array iteration connects to loops and sequences
Language: Writing array algorithms builds data manipulation skills

Digital Citizen Reminder: Apply Array Operations skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Array Operations support
On Level: Demonstrate Array Operations through practical exercises
Advanced: Apply Array Operations to solve complex problems independently
SEND: Practice Array Operations with step-by-step teacher guidance

Formative Assessment

Method: Practical Array Operations activity
CT Skill Assessed: Decomposition - breaking down Array Operations into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Array Operations. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.05 **Duration: 40 minutes**

String Operations

Learning Objectives

- Manipulate strings using concatenation, slicing, and common methods
- Apply string functions to solve text processing problems
- Understand how strings are stored as character arrays internally

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Strings are sequences of characters - understanding character codes
Science: String methods transform text - pattern recognition
Language: Writing string manipulation builds text processing skills

Digital Citizen Reminder: Apply String Operations skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with String Operations support
On Level: Demonstrate String Operations through practical exercises
Advanced: Apply String Operations to solve complex problems independently
SEND: Practice String Operations with step-by-step teacher guidance

Formative Assessment

Method: Practical String Operations activity
CT Skill Assessed: Decomposition - breaking down String Operations into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates String Operations. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.06 **Duration: 40 minutes**

Nested Loops

Learning Objectives

- Construct nested loops and trace their execution step by step
- Apply nested loops to process multi-dimensional data patterns
- Analyze the time complexity of nested loop structures

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Nested loops create 2D patterns - connecting to grid mathematics
Science: Time complexity increases with nesting level
Language: Writing nested loops builds algorithmic thinking

Digital Citizen Reminder: Apply Nested Loops skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Nested Loops support
On Level: Demonstrate Nested Loops through practical exercises
Advanced: Apply Nested Loops to solve complex problems independently
SEND: Practice Nested Loops with step-by-step teacher guidance

Formative Assessment

Method: Practical Nested Loops activity
CT Skill Assessed: Decomposition - breaking down Nested Loops into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Nested Loops. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.07 **Duration: 40 minutes**

Recursion Basics

Learning Objectives

- Explain the concept of recursion and identify base cases
- Write recursive functions to solve mathematical and logical problems
- Compare recursive and iterative approaches for the same problem

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Recursion calls itself - connecting to mathematical sequences
Science: Base cases prevent infinite recursion - logical conditions
Language: Writing recursive functions builds abstract thinking

Digital Citizen Reminder: Apply Recursion Basics skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Recursion Basics support
On Level: Demonstrate Recursion Basics through practical exercises
Advanced: Apply Recursion Basics to solve complex problems independently
SEND: Practice Recursion Basics with step-by-step teacher guidance

Formative Assessment

Method: Practical Recursion Basics activity
CT Skill Assessed: Decomposition - breaking down Recursion Basics into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Recursion Basics. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.08 **Duration: 40 minutes**

Algorithm Design

Learning Objectives

- Break down complex problems into smaller, manageable steps
- Use pseudocode and flowcharts to design algorithms before coding
- Evaluate algorithm efficiency using time and space complexity concepts

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Algorithms follow step-by-step procedures - mathematical proofs
Science: Algorithm efficiency uses Big O notation
Language: Writing algorithms builds computational thinking

Digital Citizen Reminder: Apply Algorithm Design skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Algorithm Design support
On Level: Demonstrate Algorithm Design through practical exercises
Advanced: Apply Algorithm Design to solve complex problems independently
SEND: Practice Algorithm Design with step-by-step teacher guidance

Formative Assessment

Method: Practical Algorithm Design activity
CT Skill Assessed: Decomposition - breaking down Algorithm Design into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Algorithm Design. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.09 **Duration: 40 minutes**

Problem Solving

Learning Objectives

- Apply systematic problem-solving strategies to programming challenges
- Decompose problems into subproblems and identify patterns
- Translate real-world scenarios into computational solutions

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Problem decomposition breaks complex problems into manageable parts
Science: Pattern recognition helps find similarities in problems
Language: Writing solution plans builds systematic thinking

Digital Citizen Reminder: Apply Problem Solving skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Problem Solving support
On Level: Demonstrate Problem Solving through practical exercises
Advanced: Apply Problem Solving to solve complex problems independently
SEND: Practice Problem Solving with step-by-step teacher guidance

Formative Assessment

Method: Practical Problem Solving activity
CT Skill Assessed: Decomposition - breaking down Problem Solving into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Problem Solving. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.10 **Duration: 40 minutes**

Debugging

Learning Objectives

- Identify common types of programming errors: syntax, runtime, and logic errors
- Use systematic debugging techniques to locate and fix bugs
- Apply defensive programming practices to prevent errors proactively

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Debugging uses systematic testing - scientific method
Science: Error messages provide clues for fixing code
Language: Writing bug reports builds precise communication

Digital Citizen Reminder: Apply Debugging skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Debugging support
On Level: Demonstrate Debugging through practical exercises
Advanced: Apply Debugging to solve complex problems independently
SEND: Practice Debugging with step-by-step teacher guidance

Formative Assessment

Method: Practical Debugging activity
CT Skill Assessed: Decomposition - breaking down Debugging into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Debugging. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.11 **Duration: 40 minutes**

Project - Calculator

Learning Objectives

- Design and implement a functional calculator using functions and control structures
- Apply modular programming principles by separating concerns into functions
- Handle user input, validate data, and implement error handling

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Calculator operations use arithmetic - mathematical foundations
Science: User input processing connects to input/output
Language: Writing calculator code builds practical programming skills

Digital Citizen Reminder: Apply Project - Calculator skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Project - Calculator support
On Level: Demonstrate Project - Calculator through practical exercises
Advanced: Apply Project - Calculator to solve complex problems independently
SEND: Practice Project - Calculator with step-by-step teacher guidance

Formative Assessment

Method: Practical Project - Calculator activity
CT Skill Assessed: Decomposition - breaking down Project - Calculator into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Project - Calculator. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 1.12 **Duration: 40 minutes**

Chapter 1 Review

Learning Objectives

- Consolidate understanding of functions, arrays, loops, recursion, and algorithms
- Solve 综合 problems that combine multiple programming concepts
- Reflect on learning progress and identify areas for improvement

Engage (5-7 minutes)

Teacher Activity

Show a video call in progress or recording. Ask: 'What technology makes this possible? What are the advantages over a phone call?' Discuss online meetings as a communication tool.

Student Activity

Identify features: video, screen sharing, chat, multiple participants. Discuss when video calls are better than phone calls.

Explore (10-12 minutes)

Teacher Activity

Show different online meeting tools: Zoom, Google Meet, Jitsi. Demonstrate joining a meeting, using mute/camera, screen share, chat. Compare features.

Student Activity

Observe the demonstration. Take notes on key features. Try Jitsi Meet no account needed if computers available.

Explain (10-12 minutes)

Teacher Activity

Define online meeting components: audio/video, internet connection, meeting link, host controls. Explain online workspaces: Google Workspace, Microsoft 365. Show how forms, sheets, and docs work collaboratively.

Student Activity

Take notes. Understand the difference between meetings and workspaces. See how collaboration works in real-time.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a Google Form to survey your class about their favorite subject. Include 5 questions with different input types multiple choice, text, rating. Share the form with a partner.'

Student Activity

Work independently. Create the form with varied question types. Test it by filling it out themselves. Share with partner.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. Name two online meeting tools. 2. What is the difference between a meeting and a workspace? 3. How do online workspaces support collaboration?'

Student Activity

Answer the questions. Discuss the benefits of collaborative tools.

Cross-Curricular Connections

Mathematics: Review exercises reinforce mathematical problem-solving
Science: Chapter review connects all internet and computing concepts
Language: Writing summaries builds synthesis and review skills

Digital Citizen Reminder: Apply internet safety rules you learned in every online activity.

Resources Needed:
- Review worksheet
- answer key
- chapter summary

Differentiation

Approaching: Complete fill-in-the-blank review questions
On Level: Answer short answer questions about chapter concepts
Advanced: Create a concept map connecting all chapter topics
SEND: Review key terms with teacher guidance using flashcards

Formative Assessment

Method: Chapter review quiz
CT Skill Assessed: Evaluation - self-assessing understanding of chapter concepts

Dormitory Task (Paper-and-Pencil)

Write a summary of everything you learned in this chapter. Include at least 8 key points.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.01 **Duration: 40 minutes**

Lists Introduction

Learning Objectives

- Define what a list is and explain how it differs from arrays
- Create lists, access elements, and modify list contents dynamically
- Identify situations where lists are preferred over arrays

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Lists organize data in ordered collections - sequences
Science: List indexing connects to mathematical indexing
Language: Writing list operations builds data organization skills

Digital Citizen Reminder: Apply Lists Introduction skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Lists Introduction support
On Level: Demonstrate Lists Introduction through practical exercises
Advanced: Apply Lists Introduction to solve complex problems independently
SEND: Practice Lists Introduction with step-by-step teacher guidance

Formative Assessment

Method: Practical Lists Introduction activity
CT Skill Assessed: Decomposition - breaking down Lists Introduction into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Lists Introduction. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.02 **Duration: 40 minutes**

List Operations

Learning Objectives

- Perform advanced list operations including slicing, sorting, and searching
- Use list methods to transform and analyze collections of data
- Implement algorithms that operate on lists efficiently

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: List methods add, remove, and modify elements - set operations
Science: List comprehensions use mathematical notation
Language: Writing list algorithms builds data processing skills

Digital Citizen Reminder: Apply List Operations skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with List Operations support
On Level: Demonstrate List Operations through practical exercises
Advanced: Apply List Operations to solve complex problems independently
SEND: Practice List Operations with step-by-step teacher guidance

Formative Assessment

Method: Practical List Operations activity
CT Skill Assessed: Decomposition - breaking down List Operations into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates List Operations. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.03 **Duration: 40 minutes**

List Comprehension

Learning Objectives

- Write list comprehensions to create lists concisely and efficiently
- Apply conditions and nested logic within list comprehensions
- Compare list comprehensions with traditional loops for readability and performance

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: List comprehensions use concise notation - mathematical expressions
Science: Filtering uses conditional logic - Boolean algebra
Language: Writing concise code builds efficient programming habits

Digital Citizen Reminder: Apply List Comprehension skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with List Comprehension support
On Level: Demonstrate List Comprehension through practical exercises
Advanced: Apply List Comprehension to solve complex problems independently
SEND: Practice List Comprehension with step-by-step teacher guidance

Formative Assessment

Method: Practical List Comprehension activity
CT Skill Assessed: Decomposition - breaking down List Comprehension into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates List Comprehension. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.04 **Duration: 40 minutes**

Dictionaries

Learning Objectives

- Define dictionaries as key-value pairs and explain their advantages over lists
- Create dictionaries, access values by key, and add or remove entries
- Choose appropriate data structures for different problem scenarios

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Dictionaries use key-value pairs - connecting to mapping functions
Science: Dictionary lookup is fast - understanding search efficiency
Language: Writing dictionary operations builds data modeling skills

Digital Citizen Reminder: Apply Dictionaries skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Dictionaries support
On Level: Demonstrate Dictionaries through practical exercises
Advanced: Apply Dictionaries to solve complex problems independently
SEND: Practice Dictionaries with step-by-step teacher guidance

Formative Assessment

Method: Practical Dictionaries activity
CT Skill Assessed: Decomposition - breaking down Dictionaries into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Dictionaries. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.05 **Duration: 40 minutes**

Dictionary Operations

Learning Objectives

- Perform advanced dictionary operations: iteration, merging, and comprehension
- Use dictionary methods to extract and transform data efficiently
- Solve real-world data processing problems using dictionaries

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Dictionary methods access and modify data - CRUD operations
Science: Nested dictionaries create complex data structures
Language: Writing dictionary algorithms builds data manipulation skills

Digital Citizen Reminder: Apply Dictionary Operations skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Dictionary Operations support
On Level: Demonstrate Dictionary Operations through practical exercises
Advanced: Apply Dictionary Operations to solve complex problems independently
SEND: Practice Dictionary Operations with step-by-step teacher guidance

Formative Assessment

Method: Practical Dictionary Operations activity
CT Skill Assessed: Decomposition - breaking down Dictionary Operations into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Dictionary Operations. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.06 **Duration: 40 minutes**

Tuples

Learning Objectives

- Define tuples and explain how they differ from lists in mutability and use cases
- Use tuples for function returns, dictionary keys, and data protection
- Apply tuple unpacking and understand named tuples for cleaner code

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Tuples are immutable sequences - understanding fixed data
Science: Tuple unpacking assigns multiple variables at once
Language: Writing tuple operations builds data structure awareness

Digital Citizen Reminder: Apply Tuples skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Tuples support
On Level: Demonstrate Tuples through practical exercises
Advanced: Apply Tuples to solve complex problems independently
SEND: Practice Tuples with step-by-step teacher guidance

Formative Assessment

Method: Practical Tuples activity
CT Skill Assessed: Decomposition - breaking down Tuples into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Tuples. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.07 **Duration: 40 minutes**

Sets

Learning Objectives

- Define sets and explain their properties of uniqueness and unordered storage
- Perform set operations: union, intersection, difference, and symmetric difference
- Apply sets to solve problems involving membership testing and deduplication

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Sets store unique elements - mathematical set theory
Science: Set operations include union, intersection, difference
Language: Writing set operations builds mathematical computing skills

Digital Citizen Reminder: Apply Sets skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Sets support
On Level: Demonstrate Sets through practical exercises
Advanced: Apply Sets to solve complex problems independently
SEND: Practice Sets with step-by-step teacher guidance

Formative Assessment

Method: Practical Sets activity
CT Skill Assessed: Decomposition - breaking down Sets into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Sets. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.08 **Duration: 40 minutes**

Data Organization

Learning Objectives

- Choose appropriate data structures for different types of problems
- Organize data efficiently using combinations of lists, dictionaries, and tuples
- Design data models that represent real-world entities and relationships

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Organizing data uses appropriate structures - classification
Science: Data cleaning prepares data for analysis - quality control
Language: Writing data organization plans builds systematic thinking

Digital Citizen Reminder: Apply Data Organization skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Data Organization support
On Level: Demonstrate Data Organization through practical exercises
Advanced: Apply Data Organization to solve complex problems independently
SEND: Practice Data Organization with step-by-step teacher guidance

Formative Assessment

Method: Practical Data Organization activity
CT Skill Assessed: Decomposition - breaking down Data Organization into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Data Organization. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.09 **Duration: 40 minutes**

Nested Structures

Learning Objectives

- Work with nested data structures: lists of dictionaries, dictionaries of lists, and deeper nesting
- Navigate and manipulate complex nested data efficiently
- Design hierarchical data models for real-world applications

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Nested structures contain other structures - tree concepts
Science: Accessing nested data uses chained indexing
Language: Writing nested structure operations builds complex data skills

Digital Citizen Reminder: Apply Nested Structures skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Nested Structures support
On Level: Demonstrate Nested Structures through practical exercises
Advanced: Apply Nested Structures to solve complex problems independently
SEND: Practice Nested Structures with step-by-step teacher guidance

Formative Assessment

Method: Practical Nested Structures activity
CT Skill Assessed: Decomposition - breaking down Nested Structures into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Nested Structures. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.10 **Duration: 40 minutes**

Data Processing

Learning Objectives

- Process collections of data using iteration, filtering, and aggregation
- Apply statistical operations to datasets using appropriate data structures
- Transform raw data into meaningful information through structured processing

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Data processing transforms raw data into useful information
Science: Processing pipelines connect multiple operations
Language: Writing data processing code builds ETL skills

Digital Citizen Reminder: Apply Data Processing skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Data Processing support
On Level: Demonstrate Data Processing through practical exercises
Advanced: Apply Data Processing to solve complex problems independently
SEND: Practice Data Processing with step-by-step teacher guidance

Formative Assessment

Method: Practical Data Processing activity
CT Skill Assessed: Decomposition - breaking down Data Processing into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Data Processing. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.11 **Duration: 40 minutes**

Project - Student Database

Learning Objectives

- Design and implement a complete student database system using multiple data structures
- Apply CRUD operations: create, read, update, and delete student records
- Build search, reporting, and data export features with proper error handling

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Database projects apply CRUD operations
Science: Real-world databases serve practical purposes
Language: Writing database documentation builds technical skills

Digital Citizen Reminder: Apply Project - Student Database skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Project - Student Database support
On Level: Demonstrate Project - Student Database through practical exercises
Advanced: Apply Project - Student Database to solve complex problems independently
SEND: Practice Project - Student Database with step-by-step teacher guidance

Formative Assessment

Method: Practical Project - Student Database activity
CT Skill Assessed: Decomposition - breaking down Project - Student Database into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Project - Student Database. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 2.12 **Duration: 40 minutes**

Chapter 2 Review

Learning Objectives

- Consolidate understanding of all data structures: lists, dictionaries, tuples, and sets
- Solve 综合 problems that require selecting and combining multiple data structures
- Evaluate design decisions and justify data structure choices for given problems

Engage (5-7 minutes)

Teacher Activity

Show a 2D image and its 3D-effect version. Ask: 'What makes the second image look three-dimensional?' Discuss shadows, highlights, depth.

Student Activity

Compare images. Identify 3D cues: shadows, highlights, perspective. Discuss how these create depth perception.

Explore (10-12 minutes)

Teacher Activity

Open GIMP. Demonstrate: add text layer, create layer group, apply layer mask, add shadow effect for 3D look. Students follow along.

Student Activity

Follow along. Create text with shadow effect. Experiment with layer masks. Apply 3D effects.

Explain (10-12 minutes)

Teacher Activity

Define 3D concepts: three dimensions length, width, depth. 3D text technique: duplicate layer, offset, darken. Layer masks: control visibility without deleting. Layer groups: organize complex projects.

Student Activity

Take notes. Understand how 2D images can simulate 3D. Practice layer mask concepts.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a 3D text effect for a school poster. Add shadow, highlight, and depth to make text pop.' Show examples of professional 3D text.

Student Activity

Work independently. Apply all techniques. Get peer feedback on the 3D effect quality.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What makes a 2D image look 3D? 2. What is a layer mask? 3. Name two uses of layer groups.'

Student Activity

Answer the questions. Show their work to a partner.

Cross-Curricular Connections

Mathematics: Review exercises consolidate programming concepts
Science: Chapter summaries connect related topics
Language: Writing reviews builds synthesis skills

Digital Citizen Reminder: Apply Chapter 2 Review skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Chapter 2 Review support
On Level: Demonstrate Chapter 2 Review through practical exercises
Advanced: Apply Chapter 2 Review to solve complex problems independently
SEND: Practice Chapter 2 Review with step-by-step teacher guidance

Formative Assessment

Method: Practical Chapter 2 Review activity
CT Skill Assessed: Decomposition - breaking down Chapter 2 Review into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Chapter 2 Review. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.01 **Duration: 40 minutes**

What is a Database?

Learning Objectives

- Define a database and explain its purpose in real-world applications
- Differentiate between flat-file and relational databases

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: Databases organize data systematically - information science
Science: Database concepts connect to real-world data management
Language: Writing database descriptions builds technical vocabulary

Digital Citizen Reminder: Apply What is a Database? skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with What is a Database? support
On Level: Demonstrate What is a Database? through practical exercises
Advanced: Apply What is a Database? to solve complex problems independently
SEND: Practice What is a Database? with step-by-step teacher guidance

Formative Assessment

Method: Practical What is a Database? activity
CT Skill Assessed: Decomposition - breaking down What is a Database? into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates What is a Database?. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.02 **Duration: 40 minutes**

Database Concepts

Learning Objectives

- Explain the concepts of tables, records, fields, and primary keys
- Describe how relationships connect data across tables
- Distinguish between data types used in database fields

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: Key concepts include tables, fields, and records
Science: Relational databases use mathematical relationships
Language: Writing concept explanations builds understanding

Digital Citizen Reminder: Apply Database Concepts skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Database Concepts support
On Level: Demonstrate Database Concepts through practical exercises
Advanced: Apply Database Concepts to solve complex problems independently
SEND: Practice Database Concepts with step-by-step teacher guidance

Formative Assessment

Method: Practical Database Concepts activity
CT Skill Assessed: Decomposition - breaking down Database Concepts into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Database Concepts. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.03 **Duration: 40 minutes**

SQL Introduction

Learning Objectives

- Define SQL and its role as a standard database language
- Identify the main categories of SQL commands
- Write and execute a basic SQL statement to create a table

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: SQL uses structured queries - mathematical logic
Science: SQL statements follow specific syntax rules
Language: Writing SQL queries builds database skills

Digital Citizen Reminder: Apply SQL Introduction skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with SQL Introduction support
On Level: Demonstrate SQL Introduction through practical exercises
Advanced: Apply SQL Introduction to solve complex problems independently
SEND: Practice SQL Introduction with step-by-step teacher guidance

Formative Assessment

Method: Practical SQL Introduction activity
CT Skill Assessed: Decomposition - breaking down SQL Introduction into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates SQL Introduction. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.04 **Duration: 40 minutes**

SELECT Queries

Learning Objectives

- Write SELECT statements to retrieve data from a single table
- Use SELECT DISTINCT to eliminate duplicate records
- Apply column aliases and ORDER BY to format query results

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: SELECT retrieves specific data - filtering and sorting
Science: WHERE conditions use Boolean logic
Language: Writing SELECT queries builds data retrieval skills

Digital Citizen Reminder: Apply SELECT Queries skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with SELECT Queries support
On Level: Demonstrate SELECT Queries through practical exercises
Advanced: Apply SELECT Queries to solve complex problems independently
SEND: Practice SELECT Queries with step-by-step teacher guidance

Formative Assessment

Method: Practical SELECT Queries activity
CT Skill Assessed: Decomposition - breaking down SELECT Queries into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates SELECT Queries. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.05 **Duration: 40 minutes**

INSERT Data

Learning Objectives

- Write INSERT INTO statements to add single records to a table
- Insert multiple rows using a single INSERT command
- Handle common errors when inserting data into tables

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: INSERT adds new records to tables - data entry
Science: Data types must match field definitions
Language: Writing INSERT statements builds data management skills

Digital Citizen Reminder: Apply INSERT Data skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with INSERT Data support
On Level: Demonstrate INSERT Data through practical exercises
Advanced: Apply INSERT Data to solve complex problems independently
SEND: Practice INSERT Data with step-by-step teacher guidance

Formative Assessment

Method: Practical INSERT Data activity
CT Skill Assessed: Decomposition - breaking down INSERT Data into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates INSERT Data. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.06 **Duration: 40 minutes**

UPDATE Data

Learning Objectives

- Write UPDATE statements to modify existing records
- Use WHERE clauses to target specific rows for updates
- Understand the risks of updating without a WHERE condition

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: UPDATE modifies existing records - data maintenance
Science: WHERE conditions target specific records
Language: Writing UPDATE statements builds data modification skills

Digital Citizen Reminder: Apply UPDATE Data skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with UPDATE Data support
On Level: Demonstrate UPDATE Data through practical exercises
Advanced: Apply UPDATE Data to solve complex problems independently
SEND: Practice UPDATE Data with step-by-step teacher guidance

Formative Assessment

Method: Practical UPDATE Data activity
CT Skill Assessed: Decomposition - breaking down UPDATE Data into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates UPDATE Data. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.07 **Duration: 40 minutes**

DELETE Data

Learning Objectives

- Write DELETE statements to remove specific records from a table
- Use WHERE clauses to prevent accidental deletion of all data
- Explain the difference between DELETE and DROP commands

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: DELETE removes records - data cleanup
Science: WHERE conditions prevent accidental mass deletion
Language: Writing DELETE statements builds data management skills

Digital Citizen Reminder: Apply DELETE Data skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with DELETE Data support
On Level: Demonstrate DELETE Data through practical exercises
Advanced: Apply DELETE Data to solve complex problems independently
SEND: Practice DELETE Data with step-by-step teacher guidance

Formative Assessment

Method: Practical DELETE Data activity
CT Skill Assessed: Decomposition - breaking down DELETE Data into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates DELETE Data. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.08 **Duration: 40 minutes**

WHERE Conditions

Learning Objectives

- Use comparison operators to filter records in SQL queries
- Apply logical operators AND, OR, and NOT to combine conditions
- Use BETWEEN, IN, and LIKE operators for advanced filtering

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: WHERE uses comparison operators - mathematical inequalities
Science: Multiple conditions use AND/OR - Boolean algebra
Language: Writing WHERE clauses builds query precision skills

Digital Citizen Reminder: Apply WHERE Conditions skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with WHERE Conditions support
On Level: Demonstrate WHERE Conditions through practical exercises
Advanced: Apply WHERE Conditions to solve complex problems independently
SEND: Practice WHERE Conditions with step-by-step teacher guidance

Formative Assessment

Method: Practical WHERE Conditions activity
CT Skill Assessed: Decomposition - breaking down WHERE Conditions into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates WHERE Conditions. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.09 **Duration: 40 minutes**

Aggregate Functions

Learning Objectives

- Apply COUNT, SUM, AVG, MAX, and MIN functions to calculate data summaries
- Use aliases with aggregate functions for meaningful result columns
- Combine aggregate functions with WHERE clauses for filtered calculations

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: COUNT, SUM, AVG calculate statistics - mathematical aggregation
Science: Aggregate functions summarize data collections
Language: Writing aggregate queries builds data analysis skills

Digital Citizen Reminder: Apply Aggregate Functions skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Aggregate Functions support
On Level: Demonstrate Aggregate Functions through practical exercises
Advanced: Apply Aggregate Functions to solve complex problems independently
SEND: Practice Aggregate Functions with step-by-step teacher guidance

Formative Assessment

Method: Practical Aggregate Functions activity
CT Skill Assessed: Decomposition - breaking down Aggregate Functions into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Aggregate Functions. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.10 **Duration: 40 minutes**

GROUP BY

Learning Objectives

- Use GROUP BY to categorize data into meaningful groups
- Apply HAVING to filter groups based on aggregate conditions
- Combine GROUP BY with aggregate functions for group-level summaries

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: GROUP BY categorizes data - classification
Science: GROUP BY with aggregates creates summaries
Language: Writing GROUP BY queries builds data summarization skills

Digital Citizen Reminder: Apply GROUP BY skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with GROUP BY support
On Level: Demonstrate GROUP BY through practical exercises
Advanced: Apply GROUP BY to solve complex problems independently
SEND: Practice GROUP BY with step-by-step teacher guidance

Formative Assessment

Method: Practical GROUP BY activity
CT Skill Assessed: Decomposition - breaking down GROUP BY into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates GROUP BY. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.11 **Duration: 40 minutes**

Project - Library Database

Learning Objectives

- Design a relational database schema for a library management system
- Write SQL queries to perform all CRUD operations on the library data
- Generate reports using aggregate functions and GROUP BY

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: Library databases manage book records
Science: Database projects apply all SQL operations
Language: Writing database documentation builds project skills

Digital Citizen Reminder: Apply Project - Library Database skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Project - Library Database support
On Level: Demonstrate Project - Library Database through practical exercises
Advanced: Apply Project - Library Database to solve complex problems independently
SEND: Practice Project - Library Database with step-by-step teacher guidance

Formative Assessment

Method: Practical Project - Library Database activity
CT Skill Assessed: Decomposition - breaking down Project - Library Database into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Project - Library Database. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 3.12 **Duration: 40 minutes**

Chapter 3 Review

Learning Objectives

- Consolidate understanding of database concepts and SQL operations
- Demonstrate proficiency by solving 综合 SQL challenges
- Reflect on real-world applications of database management

Engage (5-7 minutes)

Teacher Activity

Show a sales report with 100 rows. Ask: 'How would you find the top 10 sales? The average? The trend over time?' Demonstrate AutoFill and charts.

Student Activity

Consider the challenge. Recognize manual methods would be too slow. Express interest in automated solutions.

Explore (10-12 minutes)

Teacher Activity

Demonstrate AutoFill: enter 1, 2, select, drag to 20. Show number series, odd numbers, days, months. Then show relative vs absolute referencing with a multiplication table.

Student Activity

Follow along. Try AutoFill with different patterns. Create a multiplication table using absolute references.

Explain (10-12 minutes)

Teacher Activity

Define referencing types: relative changes when copied, absolute $ sign, stays fixed, mixed $A1 or A$1. Explain linking: cells within same sheet, between sheets, between workbooks. Show chart types: column, bar, line, pie.

Student Activity

Take notes. Understand when to use each referencing type. See how charts visualize data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a monthly expense tracker. Use AutoFill for dates, absolute reference for tax rate, link to a summary sheet, and create a pie chart of expense categories.'

Student Activity

Work independently. Apply all techniques. Create the complete tracker with charts.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is the difference between relative and absolute referencing? 2. When would you use a pie chart vs a bar chart? 3. How do you link cells between sheets?'

Student Activity

Answer the questions. Review their spreadsheet for completeness.

Cross-Curricular Connections

Mathematics: Review exercises consolidate database concepts
Science: Chapter summaries connect SQL topics
Language: Writing reviews builds synthesis skills

Digital Citizen Reminder: Apply Chapter 3 Review skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Chapter 3 Review support
On Level: Demonstrate Chapter 3 Review through practical exercises
Advanced: Apply Chapter 3 Review to solve complex problems independently
SEND: Practice Chapter 3 Review with step-by-step teacher guidance

Formative Assessment

Method: Practical Chapter 3 Review activity
CT Skill Assessed: Decomposition - breaking down Chapter 3 Review into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Chapter 3 Review. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.01 **Duration: 40 minutes**

How the Web Works

Learning Objectives

- Explain the client-server model and how web pages are delivered
- Describe the role of IP addresses, DNS, and HTTP in web communication
- Differentiate between static and dynamic web pages

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: The web uses client-server architecture - networking
Science: HTTP protocols govern web communication
Language: Writing web explanations builds technical understanding

Digital Citizen Reminder: Apply How the Web Works skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with How the Web Works support
On Level: Demonstrate How the Web Works through practical exercises
Advanced: Apply How the Web Works to solve complex problems independently
SEND: Practice How the Web Works with step-by-step teacher guidance

Formative Assessment

Method: Practical How the Web Works activity
CT Skill Assessed: Decomposition - breaking down How the Web Works into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates How the Web Works. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.02 **Duration: 40 minutes**

HTML Introduction

Learning Objectives

- Write a basic HTML document using proper structure and syntax
- Explain the purpose of DOCTYPE, html, head, and body tags
- Use a text editor and browser to create and preview web pages

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: HTML uses tags to structure content - markup concepts
Science: HTML documents follow specific syntax rules
Language: Writing HTML builds web development skills

Digital Citizen Reminder: Apply HTML Introduction skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with HTML Introduction support
On Level: Demonstrate HTML Introduction through practical exercises
Advanced: Apply HTML Introduction to solve complex problems independently
SEND: Practice HTML Introduction with step-by-step teacher guidance

Formative Assessment

Method: Practical HTML Introduction activity
CT Skill Assessed: Decomposition - breaking down HTML Introduction into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates HTML Introduction. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.03 **Duration: 40 minutes**

Text Elements

Learning Objectives

- Use heading, paragraph, and formatting tags to structure text content
- Apply semantic HTML tags to convey meaning and improve accessibility
- Create well-organized content using lists and block-level elements

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: HTML text elements organize content - hierarchy
Science: Headings, paragraphs, and lists structure information
Language: Writing HTML text builds web content skills

Digital Citizen Reminder: Apply Text Elements skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Text Elements support
On Level: Demonstrate Text Elements through practical exercises
Advanced: Apply Text Elements to solve complex problems independently
SEND: Practice Text Elements with step-by-step teacher guidance

Formative Assessment

Method: Practical Text Elements activity
CT Skill Assessed: Decomposition - breaking down Text Elements into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Text Elements. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.04 **Duration: 40 minutes**

Links and Images

Learning Objectives

- Create hyperlinks using anchor tags with relative and absolute paths
- Embed images using img tags with proper attributes
- Use alt text and accessibility best practices for web media

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: Links connect web pages - hypertext concept
Science: Images add visual content - media integration
Language: Writing HTML links and images builds web design skills

Digital Citizen Reminder: Apply Links and Images skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Links and Images support
On Level: Demonstrate Links and Images through practical exercises
Advanced: Apply Links and Images to solve complex problems independently
SEND: Practice Links and Images with step-by-step teacher guidance

Formative Assessment

Method: Practical Links and Images activity
CT Skill Assessed: Decomposition - breaking down Links and Images into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Links and Images. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.05 **Duration: 40 minutes**

Tables

Learning Objectives

- Create HTML tables using table, tr, th, and td elements
- Use colspan and rowspan to merge cells in complex tables
- Apply semantic table structure with thead, tbody, and tfoot

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: HTML tables organize data in rows and columns
Science: Tables use specific tags for structure
Language: Writing HTML tables builds data presentation skills

Digital Citizen Reminder: Apply Tables skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Tables support
On Level: Demonstrate Tables through practical exercises
Advanced: Apply Tables to solve complex problems independently
SEND: Practice Tables with step-by-step teacher guidance

Formative Assessment

Method: Practical Tables activity
CT Skill Assessed: Decomposition - breaking down Tables into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Tables. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.06 **Duration: 40 minutes**

Forms

Learning Objectives

- Create HTML forms using the form element and action attribute
- Explain the purpose of form methods GET and POST
- Structure forms with labels, fieldsets, and legends for accessibility

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: Form design follows user interface principles - human-computer interaction
Science: Input validation prevents data errors - quality control
Language: Writing form labels and instructions builds clear communication

Digital Citizen Reminder: Forms should be intuitive - design for the user, not the designer.

Resources Needed:
- Database software
- form design guide
- UI principles

Differentiation

Approaching: Create a simple form for data entry
On Level: Design a form with formatting, labels, and validation
Advanced: Build a comprehensive form with sub-forms and calculation fields
SEND: Create a basic form with teacher guidance

Formative Assessment

Method: Form design assessment
CT Skill Assessed: Evaluation - assessing form usability and functionality

Dormitory Task (Paper-and-Pencil)

Draw a data entry form for a library. Include fields for book title, author, and availability.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.07 **Duration: 40 minutes**

Form Elements

Learning Objectives

- Implement various input types for different data collection needs
- Use select dropdowns, radio buttons, and checkboxes appropriately
- Apply validation attributes to enforce data quality at the form level

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: Forms collect user input - data entry
Science: Form elements include text, checkbox, and submit
Language: Writing HTML forms builds interactive web skills

Digital Citizen Reminder: Apply Form Elements skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Form Elements support
On Level: Demonstrate Form Elements through practical exercises
Advanced: Apply Form Elements to solve complex problems independently
SEND: Practice Form Elements with step-by-step teacher guidance

Formative Assessment

Method: Practical Form Elements activity
CT Skill Assessed: Decomposition - breaking down Form Elements into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Form Elements. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.08 **Duration: 40 minutes**

CSS Introduction

Learning Objectives

- Explain the role of CSS in separating content from presentation
- Apply inline, internal, and external CSS to style HTML elements
- Link an external stylesheet to an HTML document

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: CSS styles web pages - visual design
Science: CSS separates content from presentation
Language: Writing CSS builds web styling skills

Digital Citizen Reminder: Apply CSS Introduction skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with CSS Introduction support
On Level: Demonstrate CSS Introduction through practical exercises
Advanced: Apply CSS Introduction to solve complex problems independently
SEND: Practice CSS Introduction with step-by-step teacher guidance

Formative Assessment

Method: Practical CSS Introduction activity
CT Skill Assessed: Decomposition - breaking down CSS Introduction into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates CSS Introduction. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.09 **Duration: 40 minutes**

CSS Selectors

Learning Objectives

- Use element, class, and ID selectors to target HTML elements
- Apply attribute and pseudo-class selectors for advanced styling
- Understand selector specificity and how CSS rules are prioritized

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: CSS selectors target HTML elements - pattern matching
Science: Selector specificity determines style priority
Language: Writing CSS selectors builds precision styling skills

Digital Citizen Reminder: Apply CSS Selectors skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with CSS Selectors support
On Level: Demonstrate CSS Selectors through practical exercises
Advanced: Apply CSS Selectors to solve complex problems independently
SEND: Practice CSS Selectors with step-by-step teacher guidance

Formative Assessment

Method: Practical CSS Selectors activity
CT Skill Assessed: Decomposition - breaking down CSS Selectors into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates CSS Selectors. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.10 **Duration: 40 minutes**

CSS Properties

Learning Objectives

- Apply text, color, and background properties to style web content
- Use the box model to control spacing and layout of elements
- Implement basic positioning and display properties for page layout

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: CSS properties control visual appearance
Science: Property values follow specific syntax
Language: Writing CSS properties builds design skills

Digital Citizen Reminder: Apply CSS Properties skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with CSS Properties support
On Level: Demonstrate CSS Properties through practical exercises
Advanced: Apply CSS Properties to solve complex problems independently
SEND: Practice CSS Properties with step-by-step teacher guidance

Formative Assessment

Method: Practical CSS Properties activity
CT Skill Assessed: Decomposition - breaking down CSS Properties into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates CSS Properties. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.11 **Duration: 40 minutes**

Project - Personal Webpage

Learning Objectives

- Build a complete multi-section personal webpage using HTML and CSS
- Demonstrate proper use of semantic HTML elements and CSS selectors
- Apply responsive design principles for mobile and desktop viewing

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: Web projects combine HTML and CSS
Science: Personal webpages express creativity
Language: Writing web documentation builds project skills

Digital Citizen Reminder: Apply Project - Personal Webpage skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Project - Personal Webpage support
On Level: Demonstrate Project - Personal Webpage through practical exercises
Advanced: Apply Project - Personal Webpage to solve complex problems independently
SEND: Practice Project - Personal Webpage with step-by-step teacher guidance

Formative Assessment

Method: Practical Project - Personal Webpage activity
CT Skill Assessed: Decomposition - breaking down Project - Personal Webpage into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Project - Personal Webpage. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 4.12 **Duration: 40 minutes**

Chapter 4 Review

Learning Objectives

- Consolidate knowledge of HTML structure, elements, and CSS styling
- Solve 综合 web development challenges combining HTML and CSS
- Evaluate 网页 quality based on accessibility, semantics, and design principles

Engage (5-7 minutes)

Teacher Activity

Show a Python program that prints 'Hello World'. Then show the same in Scratch. Ask: 'What's different? Why might someone prefer text-based programming?' Discuss transition from blocks to text.

Student Activity

Compare both programs. Identify differences: syntax, typing, precision. Discuss advantages: Python runs everywhere, faster execution.

Explore (10-12 minutes)

Teacher Activity

Open Python IDLE. Type: print'Hello World'. Show variables: x = 10, printx. Show input: name = input'Name: ', print'Hello ' + name.

Student Activity

Follow along. Type each command. See the output. Try changing values and running again.

Explain (10-12 minutes)

Teacher Activity

Define Python basics: the print function, variables, the input function, data types including integer, float, string, and boolean. Show arithmetic operators. Show comparison operators. Explain indentation and colons for code blocks.

Student Activity

Take notes. Understand data types and operators. Practice writing simple expressions.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Write a program that asks for two numbers and prints their sum, difference, product, and quotient.' Show the expected output format.

Student Activity

Work independently. Write the program. Test with different inputs. Debug any errors.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What does print do? 2. What is the difference between = and ==? 3. Why does Python need indentation?'

Student Activity

Answer the questions. Show their program output to a partner.

Cross-Curricular Connections

Mathematics: Review exercises consolidate web development concepts
Science: Chapter summaries connect HTML and CSS topics
Language: Writing reviews builds synthesis skills

Digital Citizen Reminder: Apply Chapter 4 Review skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Chapter 4 Review support
On Level: Demonstrate Chapter 4 Review through practical exercises
Advanced: Apply Chapter 4 Review to solve complex problems independently
SEND: Practice Chapter 4 Review with step-by-step teacher guidance

Formative Assessment

Method: Practical Chapter 4 Review activity
CT Skill Assessed: Decomposition - breaking down Chapter 4 Review into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Chapter 4 Review. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.01 **Duration: 40 minutes**

What is an Algorithm?

Learning Objectives

- Define algorithms and explain their role in computational thinking
- Identify the key characteristics of a well-formed algorithm
- Analyze real-world processes as sequences of algorithmic steps

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Algorithms are step-by-step procedures - mathematical logic
Science: Algorithm design is fundamental to computing
Language: Writing algorithm descriptions builds computational thinking

Digital Citizen Reminder: Apply What is an Algorithm? skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with What is an Algorithm? support
On Level: Demonstrate What is an Algorithm? through practical exercises
Advanced: Apply What is an Algorithm? to solve complex problems independently
SEND: Practice What is an Algorithm? with step-by-step teacher guidance

Formative Assessment

Method: Practical What is an Algorithm? activity
CT Skill Assessed: Decomposition - breaking down What is an Algorithm? into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates What is an Algorithm?. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.02 **Duration: 40 minutes**

Flowcharts

Learning Objectives

- Interpret and construct flowcharts using standard symbols
- Map algorithmic processes to flowchart representations
- Evaluate when flowcharts are more effective than pseudocode for communication

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Flowcharts visualize algorithms - diagrammatic representation
Science: Flowchart symbols represent different operations
Language: Writing flowcharts builds visual algorithm design

Digital Citizen Reminder: Apply Flowcharts skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Flowcharts support
On Level: Demonstrate Flowcharts through practical exercises
Advanced: Apply Flowcharts to solve complex problems independently
SEND: Practice Flowcharts with step-by-step teacher guidance

Formative Assessment

Method: Practical Flowcharts activity
CT Skill Assessed: Decomposition - breaking down Flowcharts into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Flowcharts. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.03 **Duration: 40 minutes**

Pseudocode

Learning Objectives

- Write clear pseudocode to represent algorithms
- Translate between pseudocode, flowcharts, and natural language

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Pseudocode describes algorithms in plain language
Science: Pseudocode bridges human language and code
Language: Writing pseudocode builds algorithm documentation skills

Digital Citizen Reminder: Apply Pseudocode skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Pseudocode support
On Level: Demonstrate Pseudocode through practical exercises
Advanced: Apply Pseudocode to solve complex problems independently
SEND: Practice Pseudocode with step-by-step teacher guidance

Formative Assessment

Method: Practical Pseudocode activity
CT Skill Assessed: Decomposition - breaking down Pseudocode into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Pseudocode. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.04 **Duration: 40 minutes**

Sequential Algorithms

Learning Objectives

- Identify and construct sequential algorithms in real contexts
- Understand how step order affects algorithm outcomes
- Design sequential solutions for multi-step computational problems

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Sequential algorithms follow step order
Science: Sequential logic connects to mathematical proofs
Language: Writing sequential algorithms builds basic algorithmic skills

Digital Citizen Reminder: Apply Sequential Algorithms skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Sequential Algorithms support
On Level: Demonstrate Sequential Algorithms through practical exercises
Advanced: Apply Sequential Algorithms to solve complex problems independently
SEND: Practice Sequential Algorithms with step-by-step teacher guidance

Formative Assessment

Method: Practical Sequential Algorithms activity
CT Skill Assessed: Decomposition - breaking down Sequential Algorithms into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Sequential Algorithms. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.05 **Duration: 40 minutes**

Selection Algorithms

Learning Objectives

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Selection uses conditions - Boolean logic
Science: If-else structures make decisions in algorithms
Language: Writing selection algorithms builds decision-making skills

Digital Citizen Reminder: Apply Selection Algorithms skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Selection Algorithms support
On Level: Demonstrate Selection Algorithms through practical exercises
Advanced: Apply Selection Algorithms to solve complex problems independently
SEND: Practice Selection Algorithms with step-by-step teacher guidance

Formative Assessment

Method: Practical Selection Algorithms activity
CT Skill Assessed: Decomposition - breaking down Selection Algorithms into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Selection Algorithms. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.06 **Duration: 40 minutes**

Iteration Algorithms

Learning Objectives

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Iteration repeats actions - loops
Science: For and while loops serve different purposes
Language: Writing iteration algorithms builds repetition skills

Digital Citizen Reminder: Apply Iteration Algorithms skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Iteration Algorithms support
On Level: Demonstrate Iteration Algorithms through practical exercises
Advanced: Apply Iteration Algorithms to solve complex problems independently
SEND: Practice Iteration Algorithms with step-by-step teacher guidance

Formative Assessment

Method: Practical Iteration Algorithms activity
CT Skill Assessed: Decomposition - breaking down Iteration Algorithms into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Iteration Algorithms. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.07 **Duration: 40 minutes**

Sorting Algorithms

Learning Objectives

- Implement bubble sort and insertion sort algorithms
- Compare the time complexity of different sorting approaches
- Select appropriate sorting algorithms based on data characteristics

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Sorting organizes data - mathematical ordering
Science: Different sorting algorithms have different efficiencies
Language: Writing sorting algorithms builds data organization skills

Digital Citizen Reminder: Apply Sorting Algorithms skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Sorting Algorithms support
On Level: Demonstrate Sorting Algorithms through practical exercises
Advanced: Apply Sorting Algorithms to solve complex problems independently
SEND: Practice Sorting Algorithms with step-by-step teacher guidance

Formative Assessment

Method: Practical Sorting Algorithms activity
CT Skill Assessed: Decomposition - breaking down Sorting Algorithms into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Sorting Algorithms. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.08 **Duration: 40 minutes**

Searching Algorithms

Learning Objectives

- Implement linear search and binary search algorithms
- Analyze the performance advantage of binary search on sorted data
- Determine which search algorithm to use based on data structure and requirements

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Searching finds data in collections
Science: Linear and binary search have different complexities
Language: Writing searching algorithms builds data retrieval skills

Digital Citizen Reminder: Apply Searching Algorithms skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Searching Algorithms support
On Level: Demonstrate Searching Algorithms through practical exercises
Advanced: Apply Searching Algorithms to solve complex problems independently
SEND: Practice Searching Algorithms with step-by-step teacher guidance

Formative Assessment

Method: Practical Searching Algorithms activity
CT Skill Assessed: Decomposition - breaking down Searching Algorithms into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Searching Algorithms. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.09 **Duration: 40 minutes**

Algorithm Efficiency

Learning Objectives

- Explain time and space complexity using Big-O notation
- Compare algorithms by analyzing their growth rates
- Apply complexity analysis to choose efficient solutions for given problems

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Efficiency measures algorithm performance
Science: Big O notation describes time complexity
Language: Writing efficiency analysis builds optimization skills

Digital Citizen Reminder: Apply Algorithm Efficiency skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Algorithm Efficiency support
On Level: Demonstrate Algorithm Efficiency through practical exercises
Advanced: Apply Algorithm Efficiency to solve complex problems independently
SEND: Practice Algorithm Efficiency with step-by-step teacher guidance

Formative Assessment

Method: Practical Algorithm Efficiency activity
CT Skill Assessed: Decomposition - breaking down Algorithm Efficiency into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Algorithm Efficiency. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.10 **Duration: 40 minutes**

Problem Decomposition

Learning Objectives

- Break complex problems into manageable sub-problems
- Identify dependencies and relationships between sub-problems
- Apply top-down and bottom-up decomposition strategies

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Decomposition breaks complex problems into parts
Science: Decomposition is a fundamental CT skill
Language: Writing decomposition plans builds problem-solving skills

Digital Citizen Reminder: Apply Problem Decomposition skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Problem Decomposition support
On Level: Demonstrate Problem Decomposition through practical exercises
Advanced: Apply Problem Decomposition to solve complex problems independently
SEND: Practice Problem Decomposition with step-by-step teacher guidance

Formative Assessment

Method: Practical Problem Decomposition activity
CT Skill Assessed: Decomposition - breaking down Problem Decomposition into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Problem Decomposition. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.11 **Duration: 40 minutes**

Project - Algorithm Design

Learning Objectives

- Design a complete algorithmic solution for a real-world problem

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Algorithm projects apply all design concepts
Science: Real algorithms solve practical problems
Language: Writing algorithm documentation builds project skills

Digital Citizen Reminder: Apply Project - Algorithm Design skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Project - Algorithm Design support
On Level: Demonstrate Project - Algorithm Design through practical exercises
Advanced: Apply Project - Algorithm Design to solve complex problems independently
SEND: Practice Project - Algorithm Design with step-by-step teacher guidance

Formative Assessment

Method: Practical Project - Algorithm Design activity
CT Skill Assessed: Decomposition - breaking down Project - Algorithm Design into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Project - Algorithm Design. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 5.12 **Duration: 40 minutes**

Chapter 5 Review

Learning Objectives

- Consolidate understanding of all algorithmic concepts from Chapter 5
- Apply algorithmic thinking to novel, unfamiliar problems
- Evaluate and compare different algorithmic approaches to the same problem

Engage (5-7 minutes)

Teacher Activity

Show a 3D-printed object or image. Ask: 'How do you think this was designed?' Introduce Tinkercad as a free 3D design tool. Show the interface.

Student Activity

Examine the object. Guess the design process. Express interest in creating 3D models.

Explore (10-12 minutes)

Teacher Activity

Open Tinkercad. Show navigation: orbit, pan, zoom. Demonstrate: place a shape, resize, move, group with another shape.

Student Activity

Follow along. Place shapes, resize them, move them around. Group two shapes to create a new object.

Explain (10-12 minutes)

Teacher Activity

Define 3D design concepts: workplane, shapes, grouping, alignment. Explain CSG Constructive Solid Geometry: combining basic shapes to create complex objects. Show the Circuit workspace for electronics.

Student Activity

Take notes. Understand how basic shapes combine. See the Circuit workspace for Arduino projects.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Design a simple house using basic shapes: box for walls, triangle for roof, cylinder for chimney. Group and align all parts.'

Student Activity

Work independently. Build the house step by step. Apply grouping and alignment. Share screenshot.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is CSG? 2. How do you group shapes in Tinkercad? 3. Name two workspaces in Tinkercad.'

Student Activity

Answer the questions. Show their 3D model to a partner.

Cross-Curricular Connections

Mathematics: Review exercises consolidate algorithm concepts
Science: Chapter summaries connect all algorithm topics
Language: Writing reviews builds synthesis skills

Digital Citizen Reminder: Apply Chapter 5 Review skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Chapter 5 Review support
On Level: Demonstrate Chapter 5 Review through practical exercises
Advanced: Apply Chapter 5 Review to solve complex problems independently
SEND: Practice Chapter 5 Review with step-by-step teacher guidance

Formative Assessment

Method: Practical Chapter 5 Review activity
CT Skill Assessed: Decomposition - breaking down Chapter 5 Review into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Chapter 5 Review. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.01 **Duration: 40 minutes**

What is AI?

Learning Objectives

- Define artificial intelligence and distinguish between narrow and general AI
- Identify characteristics that define intelligent behavior in machines
- Evaluate current AI capabilities against human cognitive abilities

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: AI simulates human intelligence - computer science
Science: AI uses algorithms to learn from data
Language: Writing AI explanations builds technical vocabulary

Digital Citizen Reminder: Apply What is AI? skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with What is AI? support
On Level: Demonstrate What is AI? through practical exercises
Advanced: Apply What is AI? to solve complex problems independently
SEND: Practice What is AI? with step-by-step teacher guidance

Formative Assessment

Method: Practical What is AI? activity
CT Skill Assessed: Decomposition - breaking down What is AI? into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates What is AI?. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.02 **Duration: 40 minutes**

Machine Learning Basics

Learning Objectives

- Explain the fundamental concept of learning from data
- Distinguish between supervised, unsupervised, and reinforcement learning
- Analyze real-world datasets to identify learning task types

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: ML trains models on data - statistical learning
Science: Supervised and unsupervised learning are different approaches
Language: Writing ML explanations builds scientific communication

Digital Citizen Reminder: Apply Machine Learning Basics skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Machine Learning Basics support
On Level: Demonstrate Machine Learning Basics through practical exercises
Advanced: Apply Machine Learning Basics to solve complex problems independently
SEND: Practice Machine Learning Basics with step-by-step teacher guidance

Formative Assessment

Method: Practical Machine Learning Basics activity
CT Skill Assessed: Decomposition - breaking down Machine Learning Basics into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Machine Learning Basics. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.03 **Duration: 40 minutes**

AI Applications

Learning Objectives

- Identify AI applications across diverse industries
- Analyze how AI transforms traditional processes in specific domains
- Evaluate the societal impact of AI deployment in everyday life

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: AI is used in healthcare, transportation, and more
Science: AI applications solve real-world problems
Language: Writing about AI applications builds research skills

Digital Citizen Reminder: Apply AI Applications skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with AI Applications support
On Level: Demonstrate AI Applications through practical exercises
Advanced: Apply AI Applications to solve complex problems independently
SEND: Practice AI Applications with step-by-step teacher guidance

Formative Assessment

Method: Practical AI Applications activity
CT Skill Assessed: Decomposition - breaking down AI Applications into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates AI Applications. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.04 **Duration: 40 minutes**

Internet of Things

Learning Objectives

- Define IoT and identify its core architectural components
- Explain how sensors, connectivity, and data processing work together in IoT systems
- Design a basic IoT system for a practical application

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: IoT connects physical devices to the internet
Science: IoT uses sensors and networks - physics and networking
Language: Writing about IoT builds technical description skills

Digital Citizen Reminder: Apply Internet of Things skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Internet of Things support
On Level: Demonstrate Internet of Things through practical exercises
Advanced: Apply Internet of Things to solve complex problems independently
SEND: Practice Internet of Things with step-by-step teacher guidance

Formative Assessment

Method: Practical Internet of Things activity
CT Skill Assessed: Decomposition - breaking down Internet of Things into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Internet of Things. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.05 **Duration: 40 minutes**

IoT in Agriculture

Learning Objectives

- Analyze how IoT technology addresses challenges in modern agriculture
- Evaluate the economic and environmental benefits of precision farming
- Design an IoT solution for a specific agricultural problem

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: IoT monitors crops and livestock - agricultural science
Science: Sensors collect data for precision farming
Language: Writing about IoT in agriculture builds domain knowledge

Digital Citizen Reminder: Apply IoT in Agriculture skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with IoT in Agriculture support
On Level: Demonstrate IoT in Agriculture through practical exercises
Advanced: Apply IoT in Agriculture to solve complex problems independently
SEND: Practice IoT in Agriculture with step-by-step teacher guidance

Formative Assessment

Method: Practical IoT in Agriculture activity
CT Skill Assessed: Decomposition - breaking down IoT in Agriculture into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates IoT in Agriculture. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.06 **Duration: 40 minutes**

Cloud Computing

Learning Objectives

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: Cloud computing provides on-demand resources
Science: Cloud services use internet infrastructure
Language: Writing about cloud computing builds technology awareness

Digital Citizen Reminder: Apply Cloud Computing skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Cloud Computing support
On Level: Demonstrate Cloud Computing through practical exercises
Advanced: Apply Cloud Computing to solve complex problems independently
SEND: Practice Cloud Computing with step-by-step teacher guidance

Formative Assessment

Method: Practical Cloud Computing activity
CT Skill Assessed: Decomposition - breaking down Cloud Computing into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Cloud Computing. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.07 **Duration: 40 minutes**

Cloud Services

Learning Objectives

- Compare major cloud platforms and their service offerings
- Identify appropriate cloud services for different application needs
- Evaluate cloud service reliability, security, and cost considerations

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: Cloud services include storage, computing, and software
Science: SaaS, PaaS, and IaaS are different service models
Language: Writing about cloud services builds service awareness

Digital Citizen Reminder: Apply Cloud Services skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Cloud Services support
On Level: Demonstrate Cloud Services through practical exercises
Advanced: Apply Cloud Services to solve complex problems independently
SEND: Practice Cloud Services with step-by-step teacher guidance

Formative Assessment

Method: Practical Cloud Services activity
CT Skill Assessed: Decomposition - breaking down Cloud Services into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Cloud Services. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.08 **Duration: 40 minutes**

Blockchain Basics

Learning Objectives

- Explain the fundamental principles of blockchain technology
- Describe how cryptographic hashing ensures data integrity and security
- Evaluate blockchain applications beyond cryptocurrency

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: Blockchain uses cryptographic chains - mathematical security
Science: Decentralized systems distribute trust
Language: Writing about blockchain builds emerging technology awareness

Digital Citizen Reminder: Apply Blockchain Basics skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Blockchain Basics support
On Level: Demonstrate Blockchain Basics through practical exercises
Advanced: Apply Blockchain Basics to solve complex problems independently
SEND: Practice Blockchain Basics with step-by-step teacher guidance

Formative Assessment

Method: Practical Blockchain Basics activity
CT Skill Assessed: Decomposition - breaking down Blockchain Basics into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Blockchain Basics. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.09 **Duration: 40 minutes**

5G Technology

Learning Objectives

- Explain 5G technical characteristics and how they differ from previous generations
- Analyze how 5G enables new technologies and applications
- Evaluate the societal and economic implications of 5G deployment

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: 5G uses higher frequencies - physics of waves
Science: 5G enables faster communication - engineering
Language: Writing about 5G builds technology literacy

Digital Citizen Reminder: Apply 5G Technology skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with 5G Technology support
On Level: Demonstrate 5G Technology through practical exercises
Advanced: Apply 5G Technology to solve complex problems independently
SEND: Practice 5G Technology with step-by-step teacher guidance

Formative Assessment

Method: Practical 5G Technology activity
CT Skill Assessed: Decomposition - breaking down 5G Technology into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates 5G Technology. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.10 **Duration: 40 minutes**

Ethics in Technology

Learning Objectives

- Identify ethical dilemmas arising from emerging technologies
- Apply ethical frameworks to evaluate technology decisions
- Propose responsible technology development practices

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: Technology ethics involves rights and responsibilities
Science: Ethical frameworks guide technology use
Language: Writing about ethics builds critical thinking skills

Digital Citizen Reminder: Apply Ethics in Technology skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Ethics in Technology support
On Level: Demonstrate Ethics in Technology through practical exercises
Advanced: Apply Ethics in Technology to solve complex problems independently
SEND: Practice Ethics in Technology with step-by-step teacher guidance

Formative Assessment

Method: Practical Ethics in Technology activity
CT Skill Assessed: Decomposition - breaking down Ethics in Technology into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Ethics in Technology. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.11 **Duration: 40 minutes**

Project - Tech Research

Learning Objectives

- Conduct independent research on an emerging technology topic
- Synthesize information from multiple authoritative sources
- Present technology analysis with balanced technical and ethical perspectives

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: Research projects apply investigation skills
Science: Technology research requires credible sources
Language: Writing research reports builds academic skills

Digital Citizen Reminder: Apply Project - Tech Research skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Project - Tech Research support
On Level: Demonstrate Project - Tech Research through practical exercises
Advanced: Apply Project - Tech Research to solve complex problems independently
SEND: Practice Project - Tech Research with step-by-step teacher guidance

Formative Assessment

Method: Practical Project - Tech Research activity
CT Skill Assessed: Decomposition - breaking down Project - Tech Research into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Project - Tech Research. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson:

Lesson 6.12 **Duration: 40 minutes**

Chapter 6 Review

Learning Objectives

- Consolidate knowledge of all emerging technologies covered in Chapter 6
- Analyze the interconnected nature of modern technology ecosystems
- Evaluate emerging technologies critically with technical and ethical awareness

Engage (5-7 minutes)

Teacher Activity

Show an AI chatbot conversation or image generator output. Ask: 'Is this intelligent? How does it work?' Discuss what makes AI different from regular software.

Student Activity

Observe the AI output. Discuss: 'It seems smart but follows rules.' Brainstorm examples of AI in daily life: voice assistants, recommendations.

Explore (10-12 minutes)

Teacher Activity

Activity: 'The Human Robot' — one student gives step-by-step commands, another follows literally. Show how AI follows instructions exactly, without common sense.

Student Activity

Participate in the game. Experience how literal instructions produce unexpected results. Discuss the gap between human and AI understanding.

Explain (10-12 minutes)

Teacher Activity

Define AI: systems that learn from data to make decisions. Key concepts: data fuel, patterns learning, algorithms rules. Show simple decision tree. Explain bias: AI learns from human data, which may contain biases.

Student Activity

Take notes. Understand data → pattern → decision pipeline. See how bias enters through training data.

Elaborate (8-10 minutes)

Teacher Activity

Task: 'Create a simple decision tree on paper: Should I bring an umbrella? Branches: Is it raining? Yes → Bring it, No → Check forecast, etc.' Discuss how AI makes decisions similarly.

Student Activity

Work in pairs. Create decision trees for different scenarios: 'What should I eat for lunch?', 'Should I study now?' Present to class.

Evaluate (5-8 minutes)

Teacher Activity

Exit ticket: '1. What is AI? 2. What is bias in AI? 3. Why is data important for AI?'

Student Activity

Answer the questions. Discuss the ethical implications of AI.

Cross-Curricular Connections

Mathematics: Review exercises consolidate technology concepts
Science: Chapter summaries connect all emerging tech topics
Language: Writing reviews builds synthesis skills

Digital Citizen Reminder: Apply Chapter 6 Review skills responsibly and ethically.

Resources Needed:
- Computer with Python
- code reference card
- practice exercises

Differentiation

Approaching: Complete guided practice with Chapter 6 Review support
On Level: Demonstrate Chapter 6 Review through practical exercises
Advanced: Apply Chapter 6 Review to solve complex problems independently
SEND: Practice Chapter 6 Review with step-by-step teacher guidance

Formative Assessment

Method: Practical Chapter 6 Review activity
CT Skill Assessed: Decomposition - breaking down Chapter 6 Review into manageable steps

Dormitory Task (Paper-and-Pencil)

Write a simple Python program on paper that demonstrates Chapter 6 Review. Include comments.

Teacher Reflection (Post-Lesson)

What worked well:

What to improve:

Connection to next lesson: